Build an AI Threat Intelligence Detection Chatbot

RAG + OKF for complete beginners: setup, code, result and explanation

1. What we are building

A cybersecurity helper that reads threat intelligence notes, security advisories, and CVE playbooks, finds the right note for a query, and answers from that note. Think of it as an automated security analyst for your threat database.

The idea

Threat Notes / Uploads $\rightarrow$ cut into pieces $\rightarrow$ meaning numbers Security Query $\rightarrow$ find closest piece $\rightarrow$ check it $\rightarrow$ `show the note` $\rightarrow$ `answer`

Word	Simple meaning
RAG	Find the right threat notes first, then answer from them (like an open-book exam).
OKF	A tidy way to write threat briefs: a Markdown file with a header (`type:`), a `# Title`, and `## sections`.

2. Setup (3 minutes)

1. Open `colab.research.google.com` and sign in with your Google account.
2. Click **New notebook**.
3. You will add 7 cells. For each one: click **+ Code**, paste the code, and press the play button (or `Shift+Enter`).

4. Run the cells in order. Do not skip any.

TIP: If anything breaks, click **Runtime**, then **Restart session**, and run the cells again from Cell 1.

Cell 1. Get ready

What the code does

- Installs a free tool that understands sentence meanings and Gradio for the web interface.
- Loads basic helper modules.

n
%pip install -q sentence-transformers gradio
import os, re, glob
import numpy as np
print("ready")

RESULT

```text

ready
```

A short wait while it installs. Ignore yellow warnings.

Cell 2. Write default threat notes (OKF)

What the code does

- Makes folders called `knowledge` with sub-folders (`threats`, `vulnerabilities`, `custom_uploads`).
- Saves default threat notes as `.md` files using OKF format (header with `type:`, `# Title`, `## sections`).

```
os.makedirs("knowledge/threats", exist_ok=True)

os.makedirs("knowledge/vulnerabilities", exist_ok=True)

os.makedirs("knowledge/custom_uploads", exist_ok=True)

THREAT_NOTES = {

"knowledge/threats/ransomware_lockbit.md": """---

type: ThreatIntel

title: LockBit Ransomware

---

# LockBit Ransomware

## Indicators of Compromise

Associated file hashes include malicious payloads dropping ransom notes. Look for unusual PowerShell script execution.

## Mitigation
Isolate affected endpoints immediately, disable compromised user accounts, and restore from offline immutable backups.""",

"knowledge/vulnerabilities/cve_2026_xyz.md": """---

type: Vulnerability

title: Critical Remote Code Execution CVE-2026-XYZ

---

# Critical Remote Code Execution CVE-2026-XYZ

## Affected Systems
```

Web servers running API Gateway version 4.2 or lower.

Remediation

Apply patch version 4.3 immediately or restrict external access to port 8443 via firewall rules.""",

}

for path, text in THREAT_NOTES.items():

    open(path, "w").write(text)

    print("saved default note:", path)

RESULT
Two lines indicating saved default notes.

Cell 3. Upload custom threat files (Optional)

What the code does

- Allows uploading custom `.txt` or `.md` threat intelligence files straight into the `knowledge/custom_uploads` directory.

from google.colab import files

print("Please select your threat intelligence files (.txt or .md) to upload:")

uploaded = files.upload()

for filename in uploaded.keys():

    dest_path = os.path.join("knowledge/custom_uploads", filename)

    os.rename(filename, dest_path)

```
print(f"Successfully uploaded and stored: {dest_path}")
```

RESULT
Prompts a file selection dialog and confirms saved upload paths.

Cell 4. Validate and build index

What the code does

- Opens every note and checks that it has `type:`, a `# Title`, and at least one `## section`.
- Creates `index.md`, listing all threat notes and uploaded documents.

```
files_list = [f for f in sorted(glob.glob("knowledge/**/*.md", recursive=True))
              if os.path.basename(f) not in ("index.md", "log.md")]

txt_files = glob.glob("knowledge/**/*.txt", recursive=True)

problems = 0

for f in files_list:
    text = open(f, encoding="utf-8", errors="ignore").read()
    missing = []
    if "type:" not in text: missing.append("a type: line")
    if not re.search(r"^# .+", text, re.M): missing.append("a # Title")
    if "## " not in text: missing.append("at least one ## section")
    if missing:
        problems += 1
```

```
        print("PROBLEM", f, "is missing", ", ".join(missing))
    else:
        print("OK     ", f)
index = "---\ntype: Index\ntitle: Threat Intel Index\n---\n# Threat Intel Index\n" + "".join(
    f"- {f.replace('knowledge/', '')}\n" for f in files_list + txt_files)
open("knowledge/index.md", "w").write(index)
print("\nAll documents verified & index updated!")
```

RESULT
`OK` for every valid note and confirmation of updated `index.md`.

Cell 5. Cut into pieces and search by meaning

What the code does

- **Cut:** splits each note at every `## heading`.
- **Meaning numbers:** a free embedding model converts text segments into mathematical vectors.
- **Find:** matches user queries to the closest threat segment using cosine similarity.

```
from sentence_transformers import SentenceTransformer
def load_pieces():
    pieces = []
    for path in sorted(glob.glob("knowledge/**/*.md", recursive=True)):
        if os.path.basename(path) in ("index.md", "log.md"):
            continue
```

```
    text = open(path, encoding="utf-8", errors="ignore").read()
    title_match = re.search(r"^# (.+)$", text, re.M)
    title = title_match.group(1) if title_match else os.path.basename(path)
    parts = text.split("## ")
    if len(parts) > 1:
        for part in parts[1:]:
            pieces.append({
                "file": path,
                "text": f"{title}: {part.strip()}",
                "status": "draft" if "status: draft" in text else ""
            })
    else:
        pieces.append({"file": path, "text": text, "status": ""})

for path in sorted(glob.glob("knowledge/**/*.txt", recursive=True)):
    text = open(path, encoding="utf-8", errors="ignore").read()
    pieces.append({"file": path, "text": text, "status": ""})

return pieces
```

```
PIECES = load_pieces()

model = SentenceTransformer("all-MiniLM-L6-v2")

VECTORS = model.encode([p["text"] for p in PIECES], normalize_embeddings=True)

def find(question):

    q = model.encode([question], normalize_embeddings=True)[0]

    scores = VECTORS @ q

    i = int(np.argmax(scores))

    return float(scores[i]), PIECES[i]

print(f"Total intelligence segments indexed: {len(PIECES)}")
```

RESULT
Prints total indexed threat segments ready for querying.

Cell 6. The agent: find, check, answer

What the code does

- **FIND:** retrieves the closest threat intelligence piece.
- **CHECK:** ensures match score exceeds threshold `LIMIT`.
- **ANSWER:** synthesizes a concise security answer via local LLM or outputs source text directly.

```
LIMIT = 0.30

USE_MODEL = True

_chat = None

def write_answer(question, note):
```

```
    global _chat
    if _chat is None:
        from transformers import pipeline
        _chat = pipeline("text-generation", model="Qwen/Qwen2.5-0.5B-Instruct")
    messages = [
        {"role": "system", "content": "You are a cybersecurity threat intelligence analyst. Answer strictly using the provided threat notes in 1-2 concise sentences. If unknown, say: I don't have intelligence on this threat."},
        {"role": "user", "content": f"Threat Notes: {note}\nSecurity Query: {question}"}]
    out = _chat(messages, max_new_tokens=80, do_sample=False)[0]["generated_text"]
    return out[-1]["content"].strip()
def agent(question, show=True):
    score, best = find(question)
    if score < LIMIT:
        file, answer = None, "I could not find matching threat intelligence in my database."
    else:
        file = best["file"]
        answer = write_answer(question, best["text"]) if USE_MODEL else best["text"]
    if show:
        print("QUERY:", question)
```

```
        print("SOURCE FILE:", file, f"(relevance score {score:.2f})")

        print("INTELLIGENCE BRIEF:", answer, "\n")

    return file

agent("How do I mitigate LockBit ransomware?")
```

RESULT
Outputs QUERY, SOURCE FILE, relevance score, and INTELLIGENCE BRIEF response.

Cell 7. Interactive Gradio Chatbot Web Interface

What the code does

- Wraps the agent in a web page featuring a search input, result textboxes, and 4 quick example query buttons.

```
import gradio as gr

def ask_agent(question):

    if not question.strip():

        return "", "", "Please type a security query."

    score, best = find(question)

    if score < LIMIT:

        return "None", f"{score:.2f}", "No matching threat intelligence found."

    answer = write_answer(question, best["text"]) if USE_MODEL else best["text"]

    return f"{best['file']}\n\n{best['text']}", f"{score:.2f}", answer

with gr.Blocks(title="AI Threat Intelligence Assistant") as demo:
```

```
    gr.Markdown("# 🛡️ AI Threat Intelligence Assistant\nQuery your internal security documents, CVE playbooks, and uploaded custom files.")

    q = gr.Textbox(label="Security Query / IOC Question", placeholder="Ask something about your threat notes...")

    btn = gr.Button("Query Intelligence Database", variant="primary")

    found = gr.Textbox(label="Retrieved Source Document", lines=4)

    score_box = gr.Textbox(label="Confidence Score")

    ans = gr.Textbox(label="Synthesized Response", lines=4)



    gr.Examples([

        "How do I mitigate LockBit ransomware?",

        "What are the affected systems for CVE-2026-XYZ?",

        "What are the indicators of compromise for LockBit?",

        "How do we patch CVE-2026-XYZ?"

    ], inputs=q)



    btn.click(ask_agent, q, [found, score_box, ans])

    q.submit(ask_agent, q, [found, score_box, ans])

demo.launch(share=True, debug=False)
```

RESULT

A temporary web URL (e.g., `gradio.live`) to launch and interact with the AI Threat Intelligence Chatbot.